COMP3334 — Project

1. Project Overview

In this project, your team will design and implement a secure instant messaging (IM) tool with end-to-end encryption (E2EE). This project focuses on 1:1 private messaging only (no group chat requirement) and does not require multi-device message synchronization. You may implement the client as a desktop application and the server using any network stack (HTTP/WebSocket/gRPC, etc.). The server is assumed to be honest-but-curious: it follows the protocol correctly but may inspect and analyze all data it can access.

Your system must support the following required features:

- E2EE 1:1 (peer-to-peer) private chat
- Timed self-destruct messages
- User registration and login (password + OTP)
- Friend requests / contact management (request → accept/decline)
- Offline messaging (ciphertext store-and-forward)
- Message delivery status (at least Sent and Delivered)
- Conversation list and unread counters

2. Deadlines

Submission of required materials: 16:59 PM, April 2nd, 2026. The materials include your report, codes and presentation video.

3. Threat Model & Security Goals

3.1 Threat Model

Your design must explicitly assume the following adversaries:

- Server is honest-but-curious (HbC): it follows your protocol but may inspect databases, logs, and all application-layer data, and perform traffic analysis (timestamps, message sizes, contact graph).
- Network attacker (external): may observe traffic and attempt Man-In-The-Middle (MTIM) if transport security is misconfigured; may cause packet duplication/reordering at lower layers.
- Malicious users/clients: may create accounts, spam friend requests, send malformed messages, replay old ciphertexts, or attempt impersonation at the UI layer.

3.2 Security Goals

Given the threat model above, your system must provide the following security guarantees:

- End-to-end confidentiality against the server: the server must not be able to decrypt message contents.
- End-to-end integrity and authentication between users: attackers should not forge/modify messages undetectably for a conversation they are not part of.
- Replay resistance / de-duplication: duplicated or replayed ciphertext must not be accepted as new messages.
- Key change visibility: if a contact's identity key changes, the user must be warned and your policy must be explained.

4. Cryptography & Security Engineering Requirements

4.1 Library usage

You may choose any programming language and cryptographic libraries. However, you must use well-reviewed libraries for cryptographic primitives (encryption, signatures, key agreement, hashing, random number generation). Do not implement cryptographic primitives from scratch.

Your report must include:

- Which primitives you used (e.g., AEAD scheme, KDF, signature scheme, password hashing) and why they are appropriate.
- Which library(ies) you used (name and version), and how you invoked them securely (nonces, randomness, key sizes).

4.2 Transport security

Use TLS for client-server connections. E2EE protects message content from the server, but TLS is still required to defend against network attackers and credential theft.

5. System Architecture

Your system must include at least:

- Client application(s) implementing the E2EE logic and UI.
- Server handling registration/authentication, friend requests, public key distribution (or equivalent), message relay, and offline ciphertext queue storage.

The server must never have access to the keys required to decrypt message contents. The server may store ciphertext for offline delivery.

6. Functional Requirements

6.1 Accounts & Authentication

(R1) Registration

- Users can register with a unique identifier (e.g., username or email).

- Passwords are stored using a modern password hashing scheme with a per-user salt.
- Basic password policy and rate limiting for registration/login.

(R2) Login with Password + OTP

- Support login with password plus a second factor (OTP).
- Sessions/tokens must expire and be bound to the authenticated user.

(R3) Logout / session invalidation

- Users can log out; tokens are expired/revoked promptly.

6.2 Identity & Key Management

(R4) Per-device identity keypair

- Each client generates and stores a long-term identity keypair locally.
- The server stores only the public key(s) needed for others to initiate secure sessions.

(R5) Fingerprint / verification UI

- Show a user-visible fingerprint (or safety number) for each contact/device identity key.
- Allow the user to mark a contact as “verified” (local state is acceptable).

(R6) Key change detection

- If a contact’s identity key changes, the client must warn the user.
- Define your policy (block until re-verified, or allow with warning) and justify it.

6.3 E2EE 1:1 Messaging

(R7) Secure session establishment

You must implement a secure method for two users to establish shared secrets for messaging. This course does not require a specific design such as X3DH; you may choose any protocol that is appropriate under the HbC server model. Your report must describe the protocol, its assumptions, and the security properties it provides (and does not provide).

(R8) Message encryption and authentication

- Each message must be protected with authenticated encryption (or an equivalent encrypt-then-MAC design) to provide confidentiality and integrity.
- Bind relevant metadata using authenticated associated data (AD), such as sender/receiver identifiers, conversation ID, and message counters, so tampering is detected.

(R9) Replay protection / de-duplication

- The receiver must detect and ignore replayed or duplicated ciphertext messages (within a reasonable window defined by your design).

6.4 Timed Self-Destruct Messages

(R10) TTL / expiration policy

- Support messages that self-destruct after a configurable time duration (e.g., 30 seconds, 10 minutes).
- Include the TTL/expiry policy in authenticated metadata so it cannot be altered without detection.

(R11) Client deletion behavior

- Expired messages are removed from the UI and local storage.

(R12) Server storage behavior (best-effort)

- If the server stores offline ciphertext, it must delete ciphertext after expiry (best-effort).

Important limitations:

- Self-destruct cannot prevent screenshots, copy/paste, or a malicious client. That is fine.

6.5 Friends / Contacts

(R13) Friend request workflow

- Users must add contacts via a request → accept/decline workflow (not instant adding by default).
- Users can send friend requests by username/email/contact code.

(R14) Request lifecycle

- Receiver can accept or decline; sender can cancel; both can view pending requests.

(R15) Blocking / removing

- Users can remove friends and block users; blocked users' requests/messages are ignored.

(R16) Default anti-spam control

- By default, non-friends must not be able to send arbitrary chat messages (only friend requests), or provide an equivalent control with justification.

6.6 Message Delivery Status

Delivery indicators are common IM usability features but can leak metadata. Under the HbC server model, you may rely on the server to behave correctly, but you must define the semantics precisely and discuss metadata exposure.

(R17) Minimum delivery states

- Sent: client successfully submitted the message to the server.
- Delivered: message has reached the recipient side according to your defined semantics.

(R18) Define “Delivered” semantics

- Option A (simplest): Delivered means the server placed ciphertext into the recipient’s queue or forwarded it to the recipient’s active connection.
- Option B (stronger semantics): Delivered means the recipient client sent an acknowledgement back to the sender (recommended to protect the ack with E2EE).

(R19) Metadata disclosure statement

- State what the server learns from delivery status updates (e.g., online timing).

6.7 Offline Messaging (Ciphertext Store-and-Forward)

(R20) Offline ciphertext queue

- If the recipient is offline, the server queues messages as ciphertext and relays them when the recipient comes online.

(R21) Retention and cleanup

- Define a retention policy (e.g., delete after delivery or after max age).
- Timed self-destruct TTL must be respected best-effort for queued ciphertext.

(R22) Duplicate/replay robustness

- Clients must safely handle duplicates (e.g., from retries).
- Replay protection must prevent accepting old ciphertext as a new message.

6.8 Conversation List & Unread Counters

(R23) Conversation list

- Show a list of conversations (contacts) ordered by most recent activity, including last message time.

(R24) Unread counters

- Maintain and display an unread count per conversation.

(R25) Paging / incremental loading

- Implement basic pagination or incremental loading to avoid loading all history at once.

6.9 UI

To reduce your workload, your client application does not need a beautiful Graphical User Interface (GUI). A GUI that is necessary to be used or a CLI client is fine.

7. Security Requirements

- Secure randomness: keys/nonces come from a cryptographically secure RNG.

- Secure local storage: protect private keys and protocol state at rest (e.g., OS keychain or encrypted local storage).
- Input validation: reject malformed messages and enforce reasonable size limits.
- Transport security: use TLS for client-server communication.
- Minimal sensitive logging: do not log secrets; disable verbose debug logs by default.
- Basic abuse controls: rate limit registration/login and friend requests.

8. Report Requirements

Your report must be technical and evidence-based. Include at least:

1. Your Team's ID, your names and student IDs.
2. Abstract
3. Introduction (what you built, key security properties).
4. Threat Model & Assumptions (HbC server model; out-of-scope items).
5. Architecture (trust boundaries, data flow diagrams).
6. Protocol Design (session establishment, message formats, state machine, replay handling).
7. Cryptographic Choices & Rationale (primitives, parameters, libraries, versions).
8. Security Analysis (why server cannot learn plaintext; what metadata is exposed; limitations/trade-offs).
9. Testing & Evaluation (demonstration for key functions)
    a. Demonstration of some key functions.
    b. At least 2 Security Test Cases
        i. To verify whether your design can resist attacks.
10. Future Works
11. References (papers, standards, libraries, tutorials).

9. Code

Your code must contain all the source codes, a file that can be imported to database and **a step-by-step document about how to deploy and use your application**.

This document must be able to guide a person to deploy and run your application from a **clear** Windows 11.0 / Ubuntu Linux OS (i.e., no assumptions on pre-installed software/libraries), i.e., your document should guide a person to install the needed software/libraries and use your application.

Your code should be well documented that is comprehensive comments and is readable.

10. Presentation Video

A team should record a 10-min presentation video to demonstrate the key designs in their system.

11. Submission

- Create a folder with the name *TeamID*
    - Put all your code in a folder with the name *code*
    - Rename your report with the name *report* (with the extension name, such as *pdf*)
    - Rename your video with the name *video* (with the extension name, such as *mp4*)
    - Put *code*, *report* and *video* in the folder *TeamID*
    - You should replace *TeamID* with your actual team's ID, which is in the roster.
- Compress this folder as one zip file.
- Follow the example below to name your zip file by replacing *TeamID* with your actual team's name:
    - *TeamID.zip*

We will create a Blackboard group for every team. You do not need to create a group on your own. A group assignment will be created for every group. You will be notified when we finish this work.

Please submit *TeamID.zip* to Blackboard via the group assignment. Everyone in a team can upload it on behalf of the entire team.

12. Grading Rubrics

- Code (30%)
- Report (50%)
- Presentation (20%)

Outcome Presentations	%	**A+/A/A-**	**B+/B/B-**	**C+/C/C-**	**D+/D**	**F**
Code	30%	Programs are well-organized, making good use of whitespace and comments. Variables have helpful names.	Programs are well-organized, easy to read and understand.	Programs can be read and are in a logical order.	Programs are runnable but barely readable.	Absent
Report	50%	Excellent, comprehensive and in-depth analysis with concrete facts/evidence	Clear analysis with good analysis supported by plenty of facts/evidence	Basic analysis with some level of facts/evidence	Barely relevant analysis with minimal facts/evidence	Absent
Presentation	20%	Very clear and logical	Good, easy to follow	Understandable, structured	Barely understandable	Absent

13. Academic Integrity & Licensing

- Cite external resources and libraries.
- Plagiarism or undisclosed copying violates course policy.